

Senior Android Engineering Knowledge Base

Part 2

Kotlin type system и coroutine synchronization

Theory

Сильный senior на Kotlin для Android не просто “знает generics”, а понимает, где язык создает ложное ощущение безопасности. `out` и `in` убирают целые классы ошибок, но только если вы действительно мыслите через producer/consumer; `Array<T>` специально инвариантен именно потому, что одновременно и читает, и пишет. `inline` и `reified` удобны, но у публичного API они создают отдельный класс binary-compatibility рисков: тело inline-функции вшивается в call site, а значит эволюция библиотеки требует осторожности и иногда `@PublishedApi`. Extension-функции в Kotlin не виртуальные: они резолвятся по declared type, а member-функции имеют приоритет над extension с той же сигнатурой. Это часто ломает “красивые” DSL и utility-API на границе модулей. Отдельно опасны `data class` с массивами: auto-generated `equals()` / `hashCode()` берут свойства из primary constructor, но для массивов нужно `contentEquals()` / `contentHashCode()`, иначе вы легко получаете корректный на вид, но логически неправильный key для DiffUtil, cache key или immutable MVI state. В конкурентном коде `Mutex` и `Semaphore` нужны не потому, что они “современные”, а потому что они suspend-friendly и не блокируют thread; при этом `Mutex` non-reentrant, а `Semaphore(1)` не равен по semantics привычному `@Synchronized` один-в-один. `Channel` — это не “старый Flow”, а owned queue с close/backpressure semantics и single-consumer flavor, который иногда точнее выражает задачу, чем `Flow`. ¹

Вопрос

Почему extension-функция на интерфейсе или базовом классе не может заменить polymorphism, и где это реально кусает

Junior answer. Extension-функции в Kotlin вызываются как будто это методы объекта, но на самом деле это просто синтаксический sugar. Они удобны для helper-логики и чтения кода. ²

Middle answer. Extension резолвится статически по declared type, а не по runtime type объекта. Если переменная имеет тип `Shape`, то вызов `shape.someExt()` пойдет в `Shape.someExt()`, даже если внутри лежит `Rectangle`; если у класса есть member с той же сигнатурой, победит member. ²

Senior answer. Это ловушка архитектурного уровня: extension нельзя использовать как настоящий extension point между feature-модулями или для domain-level polymorphism. Если вы строите MVI reducers, navigation contracts или mapping-слой на extension-ax и ожидаете override-like behavior, получите “работает в тестовом happy path, ломается на границе declared type” — особенно после рефакторинга, когда тип переменной обобщили до интерфейса. ³

Why this question matters. Вопрос проверяет, различает ли кандидат syntactic convenience и dispatch semantics. Поверхностный инженер любит extension как “красивые методы”, а опытный понимает, где они безопасны только как pure helper API и где ими нельзя кодировать runtime-поведение. 2

Вопрос

Почему `data class` с `ByteArray` или `Array<T>` в `primary constructor` — это скрытая production-проблема

Junior answer. `data class` автоматически генерирует `equals()` и `hashCode()`. Обычно это удобно для state и моделей. 4

Middle answer. Проблема в том, что массивы в Kotlin/JVM сравниваются не по содержимому через `==`, а как объекты, если не использовать `contentEquals()` или `contentDeepEquals()`. Значит два экземпляра `data class` с одинаковыми байтами могут оказаться “неравными”. 5

Senior answer. Это не косметическая мелочь, а ловушка для Compose state, MVI reducers, Room entities-кэшей, snapshot tests и map/set keys. Если массив участвует в equality, вы нарушаете смысл “state changed only when semantically changed”; в 2026-м правильнее либо хранить immutable `List`, либо инкапсулировать массив типом с кастомными `equals()` / `hashCode()`, либо сознательно исключить поле из `primary constructor`. 5

Why this question matters. Вопрос ловит людей, которые знают `data class` как “удобный DTO”, но не понимают контрактов equality. В Android это быстро проявляется как лишние recomposition, неверный diffing, сломанный cache key и трудновоспроизводимые баги в state machine. 5

Вопрос

Когда `Channel` действительно правильнее, чем `Flow`, и почему `Mutex` / `Semaphore` обычно лучше `synchronized` в `coroutine code`

Junior answer. `Flow` подходит для потоков данных, а `Channel` — для отправки сообщений. `Mutex` нужен для защиты общего состояния. 6

Middle answer. `Flow` — это async stream abstraction, чаще read-only API; `Channel` — owned communication primitive с `send`, `receive`, buffering и `close`, похожий на suspending queue. `Mutex.lock()` и `Semaphore.acquire()` suspend-ят coroutine, а не блокируют thread; это особенно важно на Main и на ограниченных dispatcher pool. 7

Senior answer. Правильный trap-answer такой: `Channel` нужен там, где важны queue ownership, explicit backpressure, close semantics и часто single-consumer processing — например actor mailbox, bridge к callback API или command pipeline. `Mutex` non-reentrant, а `Semaphore` fair FIFO и удобен для concurrency limits; прямолинейная замена `@Synchronized` на `Mutex` опасна, если код рассчитывал на reentrancy или thread-affinity behavior. 8

Why this question matters. Вопрос проверяет, умеет ли кандидат выбирать примитив по semantics, а не по модности. Senior-level инженер видит разницу между stream, mailbox, mutex и permit limiter и понимает, какие guarantees реально нужны конкретному экрану, reducer-актору или data pipeline. 9

Permissions и современная privacy model

Theory

Поверхностное знание permission flow сегодня легко ломается, потому что “runtime permission” больше нельзя мыслить как один диалог и один boolean. Начиная с Android 11 пользователь может выдать one-time доступ для location/mic/camera, а неиспользуемые приложения получают auto-reset sensitive permissions и app hibernation. Android 13 добавил runtime permission `POST_NOTIFICATIONS` и разнес media storage на granular `READ_MEDIA_IMAGES`, `READ_MEDIA_VIDEO`, `READ_MEDIA_AUDIO`, при этом Google прямо рекомендует предпочитать Photo Picker, если вам не нужен широкий доступ. Android 14 добавил Selected Photos Access и permission `READ_MEDIA_VISUAL_USER_SELECTED`: пользователь может дать не “все фото”, а только выбранные. На Android 15 ключевая ловушка не в новой громкой media permission, а в том, что многие команды выдумывают platform change, которого нет: важнее грамотно поддерживать продолжающуюся picker/media split-модель. В Android 16 появился opt-in Local Network Protections: это пока подготовительный режим, но он явно сигнализирует, что implicit local network access перестает быть безопасной предпосылкой; обязательным он становится для targetSdk 37. Senior-разработчик проектирует capability-first UX: сначала минимальный доступ и системный picker, потом rationale в контексте действия, потом version-gated fallback, а не наоборот. ¹⁰

Вопрос

Нужен ли `READ_MEDIA_IMAGES` или `READ_MEDIA_VIDEO`, если пользователь просто выбирает фото для аватара

Junior answer. Да, чтобы читать фотографии из галереи, нужно попросить разрешение на фото. Это стандартный сценарий для media. ¹¹

Middle answer. Не обязательно: если use case — разовый выбор пользовательского контента, Android рекомендует использовать Photo Picker. Это снимает необходимость просить широкий media permission и уменьшает friction в UX. ¹²

Senior answer. Правильный senior-ответ начинается с отказа от broad permission по умолчанию. Для avatar/attachment screen сначала применяйте system picker, а `READ_MEDIA_*` или `READ_MEDIA_VISUAL_USER_SELECTED` используйте только если бизнес-сценарий действительно требует persistent или широкого media access; иначе вы просите больше, чем нужно, ухудшаете конверсию и усложняете compliance. ¹³

Why this question matters. Это быстрый фильтр на 2026-актуальность. Кандидат, который автоматически тянется к storage/media permission, часто живет мышлением Android 9–10 и не проектирует permission model вокруг минимально достаточного доступа. ¹²

Вопрос

Что реально изменилось между Android 11, 13, 14, 15 и 16 в permission/privacy model, и где чаще всего ошибаются на интервью

Junior answer. На новых версиях стало больше ограничений и новых разрешений. Нужно проверять версию Android и запрашивать permissions по-разному. ¹⁴

Middle answer. Android 11 принес one-time permissions и auto-reset, Android 13 —

POST_NOTIFICATIONS и granular media permissions, Android 14 — Selected Photos Access, Android 15 продолжил развитие privacy/security platform behavior, а Android 16 ввел opt-in local network protections как подготовку к обязательной модели на Android 17. Ошибка — отвечать общими словами без конкретных platform deltas. ¹⁵

Senior answer. Главная ловушка — “изобрести” несуществующую новую runtime permission для Android 15 вместо реального знания изменений. Senior скажет, что эволюция здесь не только в новых prompt-ax, а в переходе к narrower capabilities: picker вместо broad media, selected access вместо full library, auto-reset для неиспользуемых apps и upcoming local network gating; следовательно permission abstraction должен быть capability-oriented, а не просто `if (SDK_INT >= X) request Y`. ¹⁶

Why this question matters. Вопрос проверяет не память на API names, а способность мыслить эволюцией платформы. Опытный инженер понимает, как эти изменения влияют на UX, analytics, feature gating и архитектуру permission coordinator-а в приложении. ¹⁷

Вопрос

Можно ли бесконечно просить permission, если пользователь уже отказал пару раз

Junior answer. Можно показать диалог еще раз и надеяться, что пользователь согласится. Если permission нужен, его надо продолжать запрашивать. ¹⁸

Middle answer. Начиная с Android 11 repeated denial для конкретного permission ведет к поведению, эквивалентному “don’t ask again”, поэтому простое повторение запроса перестает работать как раньше. Нужно проверять состояние через platform APIs и вести пользователя в Settings только когда это действительно уместно. ¹⁹

Senior answer. Правильная стратегия — не “давить” permission, а менять UX: сначала контекстная функция без permission, затем rationale рядом с действием, затем permission request, а после устойчивого denial — graceful degradation или settings handoff. Для upcoming local network protection это особенно важно: denial нужно рассматривать как штатную ветку продукта, а не как exceptional path. ²⁰

Why this question matters. В production именно permission fatigue бьет по retention, а не только по correctness. Ответ показывает, различает ли кандидат платформенные состояния permission и умеет ли строить безопасный UX вместо бесконечных retry loop-ов. ²¹

Memory leaks и profiling

Theory

В 2026 leak-диагностика на Android уже не сводится к “не держи Activity в singleton”. В Compose/Fragment hybrid-стеке самые частые реальные проблемы — это неправильная disposal strategy для ComposeView, lifecycle-unaware collection Flow, receiver/listener registration вне корректного owner-а и слишком долгоживущие объекты, в которые случайно затолкнули Context или UI reference. Для Fragment-hosted Compose особенно легко получить либо state loss, либо leak: дефолтный ViewCompositionStrategy.Default не всегда совпадает с lifecycle Fragment view, и для incremental migration Android рекомендует DisposeOnViewTreeLifecycleDestroyed.

LeakCanary автоматически отслеживает destroyed `Activity`, `Fragment`, fragment `View`, cleared `ViewModel` и `Service`, а дальше помогает пройти путь retained object → heap dump → leak trace → dominant reference. Memory Profiler нужен не как замена, а как второй инструмент: он показывает allocation churn, рост heap со временем и позволяет сравнить состояние после длинной пользовательской сессии. Senior умеет отделять leak от просто кеша или от нормального object graph: важен не факт, что объект есть в heap, а почему он остается strongly reachable после того, как его lifecycle уже закончился. ²²

Вопрос

Почему Compose внутри Fragment может либо течь, либо терять state, даже если код “визуально работает”

Junior answer. Потому что у Fragment и Compose разные lifecycle. Нужно просто аккуратно создавать и уничтожать `ComposeView`. ²³

Middle answer. В migration-сценариях дефолтная стратегия disposal для `ComposeView` не всегда совпадает с lifecycle fragment view. Android рекомендует переключать `ComposeView` на `DisposeOnViewTreeLifecycleDestroyed`, чтобы composition уничтожалась вместе с `viewLifecycleOwner`, а не в неподходящий момент. ²⁴

Senior answer. Это классический hybrid-stack trap: неправильная strategy приводит либо к удержанию composition дольше, чем живет view, либо к ранней disposal и потере локального UI state при detach/reattach сценариях. Senior знает, что здесь проблема не в “Compose unstable”, а в несовпадении ownership boundaries между Fragment view lifecycle и composition lifecycle. ²⁵

Why this question matters. Вопрос отделяет людей, которые реально мигрировали крупные экраны, от тех, кто только читал примеры single-activity Compose. На гибридных кодовых базах это один из самых дорогих классов багов: leak, state loss и flaky UI behavior одновременно. ²⁵

Вопрос

Почему `collect {}` на `Flow` может быть leak или duplicate-work bug, даже если вы запускаете его “в coroutine”

Junior answer. Потому что coroutine может пережить экран. Нужно отменять ее, когда экран закрывается. ²⁶

Middle answer. Если собирать `Flow` без lifecycle awareness, можно продолжать получать emissions в фоне, тратить батарею и держать ссылки на UI. Для Compose Android рекомендует `collectAsStateWithLifecycle`, а во Views/Fragments — `repeatOnLifecycle`. ²⁷

Senior answer. Настоящая ловушка глубже: даже если leak не случился, вы можете получить duplicate collectors после recreation и тем самым удвоить network/socket subscription или actor-side effects. Senior-level решение включает правильный collection owner, корректный `SharingStarted` режим на upstream и явное разделение screen state stream versus one-off effects. ²⁸

Why this question matters. Production-проблема здесь часто выглядит не как OOM, а как “экран иногда дважды отправляет события” или “сокет продолжает жить в фоне”. Ответ показывает, понимает ли кандидат lifecycle не только как отмену Job, но и как economics of subscriptions. ²⁹

Вопрос

Как вы реально используете LeakCanary и Memory Profiler вместе, а не “ставите библиотеку для галочки”

Junior answer. LeakCanary показывает leak trace, а Profiler показывает память. Потом можно понять, что именно течет. ³⁰

Middle answer. Практический flow такой: воспроизводите сценарий, ждете retained object report в LeakCanary, смотрите leak trace и suspect references; если картина неочевидна, берете heap dump в Memory Profiler после длинной сессии и проверяете, растет ли heap и какие объекты остаются. LeakCanary хорош в cause analysis, Profiler — в динамике allocations и heap growth. ³¹

Senior answer. Senior не чинит первый “подозрительный” reference, а формулирует ownership hypothesis: кто должен был отпустить ссылку и на каком lifecycle event. Дальше он валидирует fix не только локально, но и автоматизирует leak detection в UI tests/CI там, где регрессии повторяются. ³²

Why this question matters. Это вопрос на реальный operational опыт. Теория про GC мало помогает, если кандидат не умеет перейти от retained object к конкретному исправлению в коде и не различает memory churn, cache и настоящий leak. ³³

ANR, StrictMode и main-thread discipline

Theory

ANR — это не “просто main thread был занят дольше 5 секунд”. Для input dispatch timeout действительно критичен UI thread и дефолтный timeout на AOSP/Pixels составляет 5 секунд, но другие ANR-ы привязаны к другим типам работы: broadcast receiver, service timeout, content provider, `JobService` response. Более того, root cause не всегда находится на самом unresponsive thread: main thread может ждать lock, binder reply или scheduler slot при тяжелой system load. Поэтому взрослый подход такой: сначала определить тип ANR и unresponsive thread, потом установить root blocker. `StrictMode` здесь полезен, но это best-effort developer tool, а не детектор всего плохого: он хорошо ловит network/disk на main, leaked registration/closables, incorrect context use и даже explicit GC, но не гарантирует покрытие JNI, всех binder проблем или вообще всех expensive операций. Для производственных jank/ANR расследований центром тяжести стал Perfetto: FrameTimeline показывает, какие frames опоздали, thread state tracks — scheduled/runnable/blocked состояния, а binder/system_server tracks помогают увидеть, что проблема может сидеть не в вашем composable, а в synchronous IPC или lock contention. ³⁴

Вопрос

Почему ответ “ANR — это когда main thread блокируется больше чем на 5 секунд” неполон

Junior answer. Потому что ANR связан с тем, что приложение не отвечает. Обычно это происходит из-за тяжелой работы на главном потоке. ³⁵

Middle answer. Для input dispatch это близко к правде, и дефолтный timeout там действительно 5 секунд. Но ANR бывает у broadcast receiver, service, content provider и job response, а unresponsive thread зависит от типа ANR. ³⁶

Senior answer. Senior обязательно добавит две вещи: timeout ranges могут отличаться у OEM, а root cause может находиться на другом thread/process — например, main thread висит на lock или slow binder call. Поэтому расследование начинается не с “вынесите на IO dispatcher”, а с классификации ANR и поиска thread, который реально держит систему. ³⁶

Why this question matters. Это отличный anti-buzzword вопрос. Поверхностный кандидат повторит мантру про “не блокировать main”, а опытный покажет, что умеет читать ANR как системную проблему, а не только как локальный anti-pattern. ³⁶

Вопрос

Что StrictMode реально ловит, а что нет

Junior answer. StrictMode нужен, чтобы ловить disk и network на main thread. Его можно включить в debug-сборках. ³⁷

Middle answer. Кроме main-thread I/O, StrictMode умеет ловить slow calls, network, disk reads/writes, explicit GC, leaked closables, leaked registrations и некоторые object leak heuristics вроде class instance limits. Но сам Android прямо говорит, что StrictMode — best effort и не гарантирует обнаружение всех disk/network access, особенно через JNI. ³⁸

Senior answer. Правильная senior-позиция: StrictMode — это ранний smoke detector, а не суд последней инстанции. Он отлично дисциплинирует debug/dev builds и помогает вытаскивать accidental I/O и leak-patterns раньше, но performance triage для binder stalls, layout thrash или frame deadlines всегда потребует системной трассировки и часто Play vitals/Perfetto данных. ³⁹

Why this question matters. Вопрос проверяет практичность. Senior не религиозно “включает detectAll и счастлив”, а понимает границы инструмента и строит цепочку диагностики из нескольких уровней сигналов. ⁴⁰

Вопрос

Когда “jank” уже почти ANR, и как это смотреть в Perfetto

Junior answer. Если кадры рисуются долго, интерфейс начинает тормозить. Тогда нужно смотреть profiling tools. ⁴¹

Middle answer. Android различает slow frames, frozen frames и ANRs; frozen frames — это уже сотни миллисекунд, а ANR — секунды. Perfetto через FrameTimeline и system trace позволяет увидеть, где именно приложение пропустило frame deadline и какой thread или binder path это вызвал. ⁴²

Senior answer. Senior не ограничится “есть jank”: он проверит thread state main/render/binder, посмотрит `BinderProxy.transact*`, lock contention и `system_server`, а затем сопоставит это с user-visible flow. Очень часто expensive frame — это лишь симптом, а не источник: например, synchronous settings/system binder call на hot UI path. ⁴³

Why this question matters. В больших Android-приложениях performance bugs редко живут на одной строке кода. Ответ показывает, умеет ли человек мыслить трассой системы, а не только профилировщиком внутри процесса. ⁴⁴

Deep links, App Links и intent security

Theory

Deep linking в 2026 — это уже не просто “activity с ACTION_VIEW”. Verified App Links отличаются от custom URI schemes тем, что Android проверяет website association через Digital Asset Links и тем самым гарантирует, что только владелец домена может делегировать URL приложению. На Android 15 Dynamic App Links добавили серверную донастройку path/query/fragment rules через `assetlinks.json`, но важная ловушка в том, что эти динамические правила не могут расширить статическую manifest-scope — они только уточняют ее. В security-плоскости interview trap обычно в трех местах. Первое: intent redirection и unsafe deep links — если вы принимаете внешний `Intent`, парсите вложенный intent/extras и без валидации запускаете следующий компонент, вы фактически исполняете чужую навигационную волю в контексте своего app identity. Второе: `PendingIntent` mutability. С Android 12 mutability flag обязателен, и security guidance рекомендует `FLAG_IMMUTABLE` почти всегда; mutable нужен только для конкретных кейсов вроде inline reply и аналогичных сценариев. Третье: `android:exported` перестал быть optional для компонентов с intent filters, а implicit intents для unexported components и некоторые mutable implicit `PendingIntent` patterns дополнительно ужесточены на новых target SDK. ⁴⁵

Вопрос

Чем verified App Links принципиально отличаются от custom scheme deep links

Junior answer. App Links лучше, потому что они используют обычные `https` URL. Пользователю удобнее открывать такие ссылки. ⁴⁶

Middle answer. Verified App Links связаны с доменом через `assetlinks.json` и проверку сертификата, поэтому постороннее приложение не может просто зарегистрировать тот же `https` host и легально перехватить трафик. Custom schemes такой гарантии ownership не дают. ⁴⁷

Senior answer. В 2026 senior добавит, что Android 15 Dynamic App Links позволяют server-side refinement, но scope задается манифестом, а не наоборот. То есть грамотная стратегия — широкий manifest scope для домена плюс точная серверная маршрутизация; неграмотная — попытка “добавить новые пути на сервере”, которых статически нет в manifest. ⁴⁸

Why this question matters. Вопрос проверяет понимание trust model, а не только intent filter синтаксиса. Это критично и для product-routing, и для anti-hijacking defense. ⁴⁹

Вопрос

Почему ответ “если `PendingIntent` работает, можно оставить mutable” — плохой ответ

Junior answer. Mutable `PendingIntent` иногда нужен, если система или другое приложение будет что-то менять внутри intent. Если все работает, можно не трогать. ⁵⁰

Middle answer. Начиная с Android 12 mutability нужно указывать явно. Android security guidance рекомендует `FLAG_IMMUTABLE` для почти всех случаев, потому что mutable `PendingIntent` позволяет получателю дополнять незаполненные поля base intent и тем самым расширять поверхность атаки. ⁵¹

Senior answer. Senior ответит так: mutable — это не “по умолчанию ок”, а исключение под конкретный documented use case. Более того, начиная с Android 14 создание mutable `PendingIntent` с implicit intent без компонента/пакета приводит к исключению, так что здесь вопрос уже не только безопасности, но и forward compatibility с target SDK. ⁵²

Why this question matters. Это проверка security literacy. Опытный Android-инженер проектирует IPC surface с явной минимизацией mutability, а не воспринимает `PendingIntent` как безобидную обертку над `Intent`. ⁵³

Вопрос

Что реально означает `android:exported`, и почему intent redirection — это не “редкий edge case”

Junior answer. `android:exported` говорит, доступен ли компонент другим приложениям. Если компонент внутренний, ставим `false`. ⁵⁴

Middle answer. Для target Android 12+ значение `android:exported` нужно задавать явно у компонентов с intent filters. Но даже `exported=false` не спасает, если ваш экспортируемый endpoint принимает внешний intent, извлекает из него вложенный intent и blindly forwards его дальше. ⁵⁵

Senior answer. Senior смотрит на всю navigation/IPC цепочку как на capability boundary: whitelist action/data, нормализовать URI, не форвардить произвольные nested intents, использовать explicit component/package там, где это внутренний переход. Android 16 дополнительно ужесточает защиту от general intent redirection attacks, но рассчитывать на платформу как на единственный слой защиты нельзя. ⁵⁶

Why this question matters. Вопрос проверяет, думает ли кандидат границами доверия. В приложениях с deep links, auth flows и custom navigation это одна из самых реальных, а не академических уязвимостей. ⁵⁷

ViewModel и state scoping traps

Theory

На senior-уровне ViewModel — это не “штука, переживающая поворот”, а scoped state holder с очень конкретным owner lifetime. Типичный production-bug в fragment-based навигации случается, когда разработчик получает ViewModel не из того `ViewModelStoreOwner`: вместо parent fragment scope берет activity scope и получает неожиданное state sharing; вместо ожидаемого flow scope берет fragment scope и теряет состояние на замене экрана. Для Compose внутри Fragment это еще сложнее, потому что visual host, lifecycle owner и navigation owner могут не совпадать. `SavedStateHandle` тоже часто понимают неправильно: он помогает пережить system-initiated recreation, но он привязан к task stack и не спасает после force stop, удаления из recents или reboot, если задача исчезла. Кроме того, и `SavedStateHandle`, и

`rememberSaveable` сидят на `Bundle`, а значит непригодны для больших объектов и тяжелого screen state. Senior разработчик знает, что screen UI state должен снова выводиться из data layer по minimal saved key, а не сериализоваться целиком. Наконец, утечки ViewModel часто возникают не из-за самого класса, а из-за того, что в нем запускают долгоживущие subscription без корректного ownership или тащат туда объекты, живущие дольше/короче экрана, чем должен жить сам state holder. ⁵⁸

Вопрос

Почему shared ViewModel “вдруг шарится слишком широко” или наоборот “вдруг сбрасывается”

Junior answer. Потому что ViewModel надо получать из правильного owner. Если owner другой, будет другой экземпляр. ⁵⁹

Middle answer. Activity scope делит ViewModel между всеми fragment-ами activity, parent-fragment scope делит ее между parent и child, а fragment scope дает изолированный экземпляр. Ошибка в выборе `ViewModelStoreOwner` моментально меняет lifetime и границы шаринга. ⁶⁰

Senior answer. На senior-уровне это вопрос архитектуры навигации: scope должен совпадать не с тем, “откуда удобнее получить VM”, а с реальным жизненным циклом пользовательского flow. Если state нужен пока flow лежит в back stack — scope должен жить ровно столько; иначе вы либо получите phantom shared state между соседними экранами, либо лишние reload/actor restart при каждой перестройке дерева. ⁶¹

Why this question matters. Это реальный индикатор, проектировал ли человек многоэкранные flows, а не только single-screen examples. Ошибка в scope редко выглядит как compile error; она проявляется как “рандомный reset state” или “экран помнит чужие данные”. ⁶²

Вопрос

Переживает ли `SavedStateHandle` “process death” всегда

Junior answer. Да, для этого он и нужен: чтобы восстановить данные после process death. ⁶³

Middle answer. Не всегда: saved state привязан к task stack. Если приложение force-stop-нули, удалили из recents или устройство перезагрузили так, что task stack пропал, `SavedStateHandle` не восстановит данные. ⁶⁴

Senior answer. Правильный senior-ответ: `SavedStateHandle` хранит только минимальный transient UI input/state, достаточный для реконструкции экрана, а не сам экран целиком. Если данные важны beyond task lifetime, их место в Room/DataStore/remote source, после чего VM выводит state заново по сохраненному ID/filter/query. ⁶⁵

Why this question matters. Вопрос ловит людей, которые смешивают config change, process recreation и durable persistence. В production это быстро превращается в `TransactionTooLarge`, странные restore-баги и неверные ожидания продукта от “сохранения состояния”. ⁶⁶

Вопрос

Почему идея “сложу в ViewModel весь screen state, списки и complex objects, чтобы ничего не терять” опасна

Junior answer. В ViewModel можно держать состояние экрана, это нормально. Она как раз переживает configuration changes. ⁶⁷

Middle answer. ViewModel действительно хороша как screen-level state holder, но не как свалка всего подряд. Если вы пытаетесь через `SavedStateHandle` или `rememberSaveable` тащить большие объекты и списки, вы упираетесь в `Bundle` limits и риск `TransactionTooLargeException`. ⁶⁸

Senior answer. Senior разделяет durable data, derived screen state и tiny restorable UI element state. В VM живет логика и observable state; в `SavedStateHandle` — минимальные ключи/поля ввода; тяжелые данные заново подтягиваются и редьюсятся, иначе вы получаете хрупкую state restoration модель и размываете ответственность data layer. ⁶⁹

Why this question matters. Это вопрос на архитектурную зрелость. Хороший ответ показывает, что кандидат умеет моделировать lifespan состояния и не лечит каждую проблему одним контейнером. ⁶⁸

Compose accessibility и adaptive large screens

Theory

В Compose accessibility — это не “добавил `contentDescription` и готово”, а корректная semantics tree, которую читают одновременно и accessibility services, и UI tests. Многие composables уже дают sensible defaults, но как только вы строите кастомный элемент, визуальная структура перестает быть равна semantic structure. Нужно осмысленно задавать role, state, click action, traversal order и решать, где `mergeDescendants` помогает, а где, наоборот, скрывает отдельные actionable элементы. Сильный инженер умеет отлаживать semantics через Layout Inspector, TalkBack TreeDebug и `printToLog`, а также включать automated accessibility checks в Compose tests. На форм-факторах ловушка в другом: Android 16 для targetSdk 36 игнорирует orientation/resizability/aspect ratio restrictions на больших экранах по умолчанию, а Android 17 убирает opt-out. Значит старый экран, “нормально работавший в portrait phone-only мире”, теперь внезапно размазывается, имеет overlapping panes, некорректный focus traversal и неадекватные touch targets на планшетах, foldables и desktop-like окнах. Senior отвечает не “добавьте landscape layout”, а строит adaptive UI через window size classes, responsive layout decisions и при необходимости adaptive/navigation scaffolds и posture-aware patterns. В 2026 это уже не nice-to-have, а часть baseline quality для target SDK upgrades. ⁷⁰

Вопрос

Если экран визуально красивый, значит ли это, что с accessibility все уже хорошо

Junior answer. Не обязательно, потому что TalkBack читает экран по-своему. Нужно проверить основные элементы и описания. ⁷¹

Middle answer. Compose опирается на semantics, а не на “то, как вижу я глазами”. Если у кастомного элемента нет корректной semantics information, testing и accessibility services не поймут его роль, состояние и действия. ⁷²

Senior answer. Senior-level ответ включает то, что semantics tree — это отдельный контракт UI. Именно поэтому a11y и тесты часто ломаются одновременно: когда команда строит сложный composable из `Box/Row/Icon/Text` и забывает semantic role/actions, экран может выглядеть идеально, но быть практически неиспользуемым для TalkBack и нестабильным для test finders. ⁷³

Why this question matters. Вопрос проверяет, работал ли человек с accessibility как с инженерным качеством, а не как с чекбоксом. Хороший ответ почти всегда приходит из реального опыта дебага semantics tree и автоматизированных проверок. ⁷⁴

Вопрос

Всегда ли `mergeDescendants` улучшает a11y

Junior answer. Да, так экран становится проще для чтения. Меньше фокусов — меньше шума. ⁷⁵

Middle answer. Не всегда: `mergeDescendants` полезен, когда несколько визуальных кусочков составляют одну смысловую сущность. Но если внутри были отдельные действия или важные состояния, чрезмерное merge скроет их от accessibility services. ⁷⁶

Senior answer. Senior балансирует granularity и discoverability. Он объединяет metadata/text clusters, но не уничтожает отдельные interactive targets; при необходимости дополнительно задает traversal groups и traversal order, вместо того чтобы насильно схлопывать всю иерархию в один узел. ⁷⁷

Why this question matters. Это ловушка на “магическое мышление”. Поверхностный человек запомнил один модификатор, опытный понимает, что у screen reader usability есть структура, а не только количество узлов. ⁷⁸

Вопрос

Что реально начнет ломаться на существующем Compose-экране при `targetSdk 36` и больших экранах

Junior answer. Возможно, нужно будет лучше поддержать landscape и планшеты. Иногда layout выглядит растянутым. ⁷⁹

Middle answer. На Android 16 ограничения orientation/resizability/aspect ratio для больших экранов игнорируются по умолчанию, поэтому screen, рассчитанный на portrait-only phone, может внезапно занять большое окно и показать overlap, обрезание, плохой focus order и неканоничный navigation pattern. Решение — adaptive layout через window size classes и responsive UI, а не манифестные запреты. ⁸⁰

Senior answer. Senior также упомянет migration pressure: Android 16 еще позволяет временный opt-out, но Android 17 его убирает. Значит качество adaptive behavior — уже не “потом сделаем”, а

часть пути targetSdk upgrade; команды, которые этого не заложат заранее, получают не только UX bugs, но и более дорогую стабилизацию перед выпуском. ⁸¹

Why this question matters. Вопрос отделяет тех, кто действительно следит за form-factor roadmap Android, от тех, кто все еще мыслит телефоном как единственной целью. Для senior engineer это уже часть platform readiness и release planning. ⁸²

Interop, Gradle performance и инициализация приложения

AndroidView interop и миграция View → Compose на больших кодовых базах

Theory

Когда проект частично мигрирован на Compose, trap находится не только в `ComposeView` внутри `Fragment`, но и в обратной стороне — legacy `View` внутри Compose через `AndroidView`. Этот API полезен для SDK/Ads/WebView/Platform widgets и как временный bridge, но Android прямо советует не использовать его как постоянную оболочку вокруг собственных custom Views, если их можно переписать на Compose. Главные ошибки — создавать `View` не в `factory`, а в composition path; держать mutable hooks так, что `update` становится тяжелой imperative mini-render loop; не учитывать mismatch между Compose constraints и legacy view measurement; забывать про `onReset` / `onRelease` при reuse внутри `LazyColumn` и получать churn или state bleed между reused instances. На больших проектах успешная миграция обычно идет screen-by-screen и component-by-component: сначала новые экраны в Compose, затем shared design system, затем самые простые custom Views, затем сложные контейнеры. Цель — не “обернуть все старое завтра”, а постепенно переместить state ownership и presentation в UDF-модель, сохраняя совместимость на границе хостинга и не создавая новый слой tech debt под видом interop. ⁸³

Вопрос

Почему `AndroidView` в `LazyColumn` часто становится perf-bug или state-bug, хотя на маленьком списке все выглядит нормально

Junior answer. Потому что View тяжелее, чем composable, и список начинает тормозить. Нужно быть аккуратнее с количеством view. ⁸⁴

Middle answer. В lazy-контейнерах важно уметь переиспользовать underlying `View`. Для этого у `AndroidView` есть overload с `onReset` и `onRelease`; без этого вы можете получить лишние создания view или утечки/грязное состояние при reuse. ⁸⁵

Senior answer. Senior скажет, что здесь проблема не только в allocation cost, а в imperative state retention старого view-world. Если reused instance не получает корректный reset всех transient полей, у вас появляются phantom selections, неправильные listeners, stale adapters и flaky scroll bugs, которые почти не воспроизводятся на коротких списках. ⁸⁴

Why this question matters. Это вопрос на реальный interop опыт. Ответ показывает, понимает ли человек reuse semantics и lifecycle границы legacy View внутри declarative tree, а не только API сигнатуру. ⁸⁵

Вопрос

Как выглядит реалистичная стратегия миграции большого View-приложения, если нельзя “переписать все за квартал”

Junior answer. Можно добавлять Compose постепенно, экран за экраном. Главное — не пытаться переписать все сразу. ⁸⁶

Middle answer. Практично начинать с новых экранов и изолированных shared components, а interop использовать как transition layer. При этом важно рано разделить state и presentation, иначе вы просто перенесете stateful View-архитектуру под новый UI toolkit. ⁸⁷

Senior answer. Senior строит migration как организационную и техническую программу: composable design system, host strategy для Fragments, explicit rules где допустим `AndroidView`, measurable milestones по убиранию custom Views и тестовая/профилировочная валидация на каждом этапе. Иначе interop быстро становится permanent architecture, а не временным мостом. ⁸⁸

Why this question matters. Интервьюер здесь ищет не энтузиазм, а способность управлять долгой миграцией без взрыва риска. Хороший ответ всегда включает boundaries, критерии успеха и план по устранению временных решений. ⁸⁷

Gradle build performance internals

Theory

Build performance — область, где senior легко отделяется от cargo cult. Version Catalogs и convention plugins сами по себе не делают сборку быстрой; они делают ее управляемой и уменьшают вероятность хаотичного build logic. Реальный выигрыш дают configuration cache, хороший build cache hit rate, avoidance eager configuration и снижение annotation-processing overhead. Gradle прямо рекомендует configuration cache как главный ускоритель повторных сборок: он кеширует configuration phase и при неизменных inputs может вообще ее пропускать. Build cache сохраняет task outputs локально или удаленно и позволяет переиспользовать результаты между сборками; для Android-проектов Gradle прямо пишет о значимом ускорении при хорошем кешировании. KSP важен не как модный replacement, а потому что это Kotlin-first processing, который обычно быстрее kapt и лучше понимает Kotlin symbols; при этом миграция часто идет по модулям и может быть смешанной. Convention plugins полезнее `buildSrc`, когда вы хотите изолировать и переиспользовать build logic более контролируемо. Самая частая senior-ловушка: сначала мерить, где реально тратится время, а потом включать инструменты; без этого команда “оптимизирует” TOML-структуру и aliases, игнорируя configuration misses, incompatible plugins и плохой cache behavior. ⁸⁹

Вопрос

Что в реальном Android build двигает стрелку сильнее: Version Catalogs, convention plugins, KSP, configuration cache или build cache

Junior answer. Все это улучшает build и делает проект современнее. Лучше использовать весь актуальный стек. ⁹⁰

Middle answer. Для времени сборки обычно сильнее влияют configuration cache, build cache hit rate и переход от kapt к KSP там, где это поддержано. Version Catalogs и convention plugins больше

помогают со структурой и maintainability, хотя косвенно тоже улучшают дисциплину build logic.

91

Senior answer. Senior ответит “it depends, но измеримо”: если build тонет в configuration phase — configuration cache; если CI повторяет похожие builds — remote/local build cache; если проект тяжело зависит от annotation processing — KSP migration. А вот catalog aliases без профилирования — это почти всегда удобство, а не заметный перф-буст. 92

Why this question matters. Это вопрос на инженерный приоритет. Senior не путает “улучшили структуру build scripts” с “ускорили builds” и умеет выбирать рычаг по profile, а не по тренду. 93

Вопрос

Почему KSP не всегда значит “немедленно удалить kapt везде”

Junior answer. Потому что не все библиотеки поддерживают KSP. Иногда приходится оставить kapt. 94

Middle answer. Официальная миграция допускает co-existence kapt и KSP в одном проекте и module-by-module переход. Это практичный путь для крупных codebase, особенно если часть processors уже совместима, а часть еще нет. 95

Senior answer. Senior дополнит, что смысл миграции — не “убрать старое любой ценой”, а снизить processor overhead без поломки toolchain. Если критичный processor, например часть DI/database tooling, еще не готов или не стабилен в вашем стеке, смешанная стратегия лучше, чем идеологически чистая, но дорогая миграция. 96

Why this question matters. Вопрос проверяет зрелость решений под constraints реального проекта. Человек с production опытом почти всегда думает стадиями миграции, а не бинарной заменой технологий. 95

App Startup и initialization at scale

Theory

Инициализация на старте — классическая зона скрытого долга. Исторически Android-библиотеки любили собственные `ContentProvider` для auto-init, но Android прямо отмечает, что providers дороги в создании, замедляют startup и инициализируются в неопределенном порядке. Jetpack App Startup решает именно это: один `InitializationProvider`, явные `Initializer` - зависимости и возможность lazy/manual init через `AppInitializer`, если компонент не нужен на cold start. Но ловушки остаются. Во-первых, dependency graph initializers должен отражать реальные зависимости, иначе вы получаете startup-time races. Во-вторых, если вы отключили auto init у компонента, вы отключили и auto init его dependencies, что легко забыть. В-третьих, multi-process приложения: Android-процессы независимы, а provider/initializer живут в process context; на практике это означает, что наивная логика “инициализируем один раз на приложение” часто ложна. Наконец, App Startup — это не индульгенция на тяжелую работу в `create()`: Android startup guidance по-прежнему рекомендует откладывать non-essential initialization и использовать measurement tools, включая Macrobenchmark, Perfetto и Startup Profiles. Senior-level решение — минимальный eager core, measurable deferred init и явный ownership того, что обязано быть готово до первого экрана, а что нет. 97

Вопрос

Почему “каждая библиотека сама заведет свой `ContentProvider` и проинициализируется” — плохая масштабируемая стратегия

Junior answer. Потому что startup может стать медленнее. Лучше меньше тяжелой инициализации при запуске. ⁹⁸

Middle answer. Android прямо отмечает две проблемы: content providers дороги по startup cost и инициализируются в неопределенном порядке. App Startup объединяет инициализацию через один provider и позволяет явно задать зависимости между initializers. ⁹⁹

Senior answer. На масштабе это еще и governance-проблема: множество auto-init providers превращают cold start в неуправляемый набор side effects. Senior строит registry/ownership модель инициализации: что eager, что deferred, какие зависимости допустимы и как этот graph измеряется после каждого релиза. ¹⁰⁰

Why this question matters. Вопрос проверяет, умеет ли кандидат мыслить startup как системный бюджет, а не как “место, куда удобно положить init SDK”. Это очень сильно влияет на TTFF, ANR risk и release predictability. ¹⁰¹

Вопрос

Что ломается в App Startup инициализации в multi-process приложении

Junior answer. В разных процессах могут быть разные экземпляры инициализации. Нужно учитывать, где именно выполняется код. ¹⁰²

Middle answer. Android процессы независимы, а provider имеет свой process context. Если компонент вашего приложения или provider запускается в отдельном процессе, нельзя автоматически считать, что startup initialization “уже случилась” в другом процессе. ¹⁰²

Senior answer. Senior-level trap-answer: App Startup упорядочивает init внутри процесса, но не дает magically global once-per-app semantics across processes. Поэтому cross-process-sensitive SDK, shared storage locks, metrics/reporting и background-process entrypoints требуют явной process-aware стратегии, а иногда — полного отказа от eager init вне main process. ¹⁰³

Why this question matters. Вопрос быстро выявляет, сталкивался ли человек с push/service/remote-process реальностью. Те, кто работал только с простыми single-process apps, почти всегда недооценивают этот класс багов. ¹⁰⁴

Практические задания

Задание

Напишите `data class`, у которой в primary constructor есть `ByteArray`, затем покажите, почему два инстанса с одинаковым содержимым оказываются “неравными”, и исправьте модель так, чтобы equality стала семантической. Это проверяет понимание Kotlin equality contracts, hidden state bugs в MVI/Compose и аккуратность model design. ⁵

Задание

Реализуйте `PermissionCoordinator` для выбора изображения профиля: на Android 13+ он должен предпочесть Photo Picker без broad media permission, на Android 14 корректно обрабатывать selected photos model, а denial path должен переводить UI в degradable mode вместо бесконечного re-request. Это проверяет современную permission архитектуру, version gating и capability-first UX. ¹⁰⁵

Задание

Дан Fragment, который хостит `ComposeView` и собирает `Flow` в `launchWhenStarted`; найдите две проблемы, из-за которых возможны leak/state loss/duplicate collection, и перепишите его на корректные APIs. Это проверяет hybrid lifecycle boundaries, `ViewCompositionStrategy` и lifecycle-aware flow collection. ¹⁰⁶

Задание

Вам дали deep link entry activity, которая читает `Intent.EXTRA_INTENT` и immediately стартует его. Покажите безопасную переработку: explicit validation, allowlist action/data, отказ от blind forwarding и корректные manifest/security choices. Это проверяет App Links/intent security и практическое понимание redirection vulnerabilities. ¹⁰⁷

Задание

Соберите пример `AndroidView` внутри `LazyColumn`, у которого без `onReset` видно stale state при scroll/reuse, затем исправьте его. Это проверяет interop reuse semantics, view reset discipline и миграцию legacy widgets внутрь Compose. ⁸⁵

Задание

Сконструируйте мини `build-logic` через convention plugin и покажите, как вы бы измерили эффект от перехода части модулей с карт на KSP и от включения configuration cache. Комментарий к этому заданию должен объяснять не только “что включили”, но и “какой bottleneck это адресует”. Это проверяет зрелость в Gradle performance и привычку мыслить профилированием, а не чеклистом. ¹⁰⁸

1 Generics: in, out, where

https://kotlinlang.org/docs/generics.html?utm_source=chatgpt.com

2 3 Extensions | Kotlin Documentation

https://kotlinlang.org/docs/extensions.html?utm_source=chatgpt.com

4 Data classes

https://kotlinlang.org/docs/data-classes.html?utm_source=chatgpt.com

5 Equality | Kotlin Documentation

<https://kotlinlang.org/docs/equality.html>

6 7 9 Flow | kotlinx.coroutines

https://kotlinlang.org/api/kotlinx.coroutines/kotlinx-coroutines-core/kotlinx.coroutines.flow/-flow/?utm_source=chatgpt.com

8 Channel | [kotlinx.coroutines](#)

https://kotlinlang.org/api/kotlinx.coroutines/kotlinx-coroutines-core/kotlinx.coroutines.channels/-channel/?utm_source=chatgpt.com

10 15 16 19 20 21 Permissions updates in Android 11

https://developer.android.com/about/versions/11/privacy/permissions?utm_source=chatgpt.com

11 14 Permissions on Android | Privacy

https://developer.android.com/guide/topics/permissions/overview?utm_source=chatgpt.com

12 13 105 Behavior changes: Apps targeting Android 13 or higher

https://developer.android.com/about/versions/13/behavior-changes-13?utm_source=chatgpt.com

17 18 Request runtime permissions | Privacy

https://developer.android.com/training/permissions/requesting?utm_source=chatgpt.com

22 23 24 25 106 Using Compose in Views | Jetpack Compose

https://developer.android.com/develop/ui/compose/migrate/interoperability-apis/compose-in-views?utm_source=chatgpt.com

26 27 28 29 Use Kotlin coroutines with lifecycle-aware components

https://developer.android.com/topic/libraries/architecture/coroutines?utm_source=chatgpt.com

30 Introduction - LeakCanary

https://square.github.io/leakcanary/fundamentals/?utm_source=chatgpt.com

31 How LeakCanary works

https://square.github.io/leakcanary/fundamentals-how-leakcanary-works/?utm_source=chatgpt.com

32 33 Fixing a memory leak - LeakCanary

https://square.github.io/leakcanary/fundamentals-fixing-a-memory-leak/?utm_source=chatgpt.com

34 35 36 Diagnose and fix ANRs | App quality | Android Developers

<https://developer.android.com/topic/performance/anrs/diagnose-and-fix-anrs>

37 38 39 40 StrictMode | API reference | Android Developers

<https://developer.android.com/reference/kotlin/android/os/StrictMode>

41 42 Slow rendering | App quality

https://developer.android.com/topic/performance/vitals/render?utm_source=chatgpt.com

43 Find the unresponsive thread | App quality | Android Developers

<https://developer.android.com/topic/performance/anrs/find-unresponsive-thread>

44 What is Perfetto? - Perfetto Tracing Docs

https://perfetto.dev/docs/?utm_source=chatgpt.com

45 46 48 About App Links | App architecture

https://developer.android.com/training/app-links/about?utm_source=chatgpt.com

47 49 Verify App Links | App architecture

https://developer.android.com/training/app-links/verify-applinks?utm_source=chatgpt.com

50 PendingIntent | API reference

https://developer.android.com/reference/android/app/PendingIntent?utm_source=chatgpt.com

51 Behavior changes: Apps targeting Android 12

https://developer.android.com/about/versions/12/behavior-changes-12?utm_source=chatgpt.com

52 Behavior changes: Apps targeting Android 14 or higher

https://developer.android.com/about/versions/14/behavior-changes-14?utm_source=chatgpt.com

53 Pending intents | Security

https://developer.android.com/privacy-and-security/risks/pending-intent?utm_source=chatgpt.com

54 55 | App architecture

https://developer.android.com/guide/topics/manifest/activity-element?utm_source=chatgpt.com

56 57 107 Intent redirection | Security

https://developer.android.com/privacy-and-security/risks/intent-redirection?utm_source=chatgpt.com

58 59 67 ViewModel overview | App architecture

https://developer.android.com/topic/libraries/architecture/viewmodel?utm_source=chatgpt.com

60 62 Communicate with fragments | App architecture

https://developer.android.com/guide/fragments/communicate?utm_source=chatgpt.com

61 Guide to app architecture

https://developer.android.com/topic/architecture?utm_source=chatgpt.com

63 Saved State module for ViewModel (Views)

https://developer.android.com/topic/libraries/architecture/views/viewmodel/viewmodel-savedstate-views?utm_source=chatgpt.com

64 65 66 Saved State module for ViewModel | App architecture

https://developer.android.com/topic/libraries/architecture/viewmodel/viewmodel-savedstate?utm_source=chatgpt.com

68 69 Save UI state in Compose

https://developer.android.com/develop/ui/compose/state-saving?utm_source=chatgpt.com

70 72 73 74 Semantics | Jetpack Compose

https://developer.android.com/develop/ui/compose/accessibility/semantics?utm_source=chatgpt.com

71 Accessibility in Jetpack Compose

https://developer.android.com/develop/ui/compose/accessibility?utm_source=chatgpt.com

75 76 77 78 Merging and clearing | Jetpack Compose

https://developer.android.com/develop/ui/compose/accessibility/merging-clearing?utm_source=chatgpt.com

79 80 82 Behavior changes: Apps targeting Android 16 or higher

https://developer.android.com/about/versions/16/behavior-changes-16?utm_source=chatgpt.com

81 Restrictions on orientation and resizability are ignored

https://developer.android.com/about/versions/17/changes/ff-restrictions-ignored?utm_source=chatgpt.com

83 84 85 88 Using Views in Compose

https://developer.android.com/develop/ui/compose/migrate/interoperability-apis/views-in-compose?utm_source=chatgpt.com

86 87 Other considerations | Jetpack Compose

https://developer.android.com/develop/ui/compose/migrate/other-considerations?utm_source=chatgpt.com

89 91 92 108 Configuration Cache

https://docs.gradle.org/current/userguide/configuration_cache.html?utm_source=chatgpt.com

90 Version Catalogs

https://docs.gradle.org/current/userguide/version_catalogs.html?utm_source=chatgpt.com

93 Best Practices for Performance

https://docs.gradle.org/current/userguide/best_practices_performance.html?utm_source=chatgpt.com

94 Migrate from kapt to KSP

https://kotlinlang.org/docs/ksp-kapt-migration.html?utm_source=chatgpt.com

95 96 Migrate from kapt to KSP | Android Studio

https://developer.android.com/build/migrate-to-ksp?utm_source=chatgpt.com

97 100 App Startup | App architecture | Android Developers

<https://developer.android.com/topic/libraries/app-startup>

98 101 App startup time | App quality

https://developer.android.com/topic/performance/vitals/launch-time?utm_source=chatgpt.com

99 App Startup | App architecture

https://developer.android.com/topic/libraries/app-startup?utm_source=chatgpt.com

102 103 104 Processes and threads overview | App quality

https://developer.android.com/guide/components/processes-and-threads?utm_source=chatgpt.com